SIMATS ENGINEERING

ASSESSMENT TOOL 3

ARTIFICIAL INTELLIGENCE

Course Code: CSA17

Assessment Details	Information
Course Outcome	CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively (BL3).
Assessment Type	Descriptive Written Test
Weightage	30%
Duration	45 Minutes
Total Marks	20
Questions	Answer any TWO

Student Name: Y.Vyshnavi Reddy

Register Number:192412642

Code of Conduct

I Y.Vyshnavi Reddy/192412642 (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____Y.Vyshnavi Reddy_____

Q1. Breadth-First Search (BFS): Rescue Robot in a Building

Breadth-First Search (BFS) is an uninformed search algorithm that explores nodes level by level. It uses a FIFO (First In, First Out) queue. The connections are S–A, S–B, A–C, A–D, B–D, B–E, C–G and D–G. Assume neighbors are considered in the order listed and each room is marked visited when it is enqueued.

(a) BFS trace: queue contents and visit order

Step	Room expanded	Queue after expansion	Expanded/visited order
Initial	—	S	—
1	S	A, B	S
2	A	B, C, D	S, A
3	B	C, D, E	S, A, B
4	C	D, E, G	S, A, B, C
5	D	E, G	S, A, B, C, D
6	E	G	S, A, B, C, D, E
7	G	Empty	S, A, B, C, D, E, G

Note: G is discovered while expanding C, so BFS can stop when G is dequeued or, in a common implementation, when it is first discovered. The table shows the queue trace through G.

(b) Path and number of doors

With the stated neighbor order, BFS returns S → A → C → G. It uses 3 doors.

There is also an equally short path S → A → D → G, which also uses 3 doors. BFS returns the first shortest path according to the neighbor ordering.

(c) Why BFS guarantees the fewest doors

BFS explores all rooms at depth 0, then depth 1, then depth 2, and so on. Therefore, the first goal found is at the minimum depth. When every door has equal cost, minimum depth means the fewest doors.

(d) Time and space complexity

For branching factor b and shallowest goal depth d, BFS has time complexity O(b^(d+1)) and space complexity O(b^(d+1)). With b = 10 and d = 5, the order is O(10^6), or about one million nodes in the worst-case growth estimate. Memory is a major problem because BFS stores a large frontier of unexplored nodes, which grows exponentially with depth.

Q2. Depth-First Search (DFS): Finding a File in Nested Folders

(a) How DFS works and the role of the stack

Depth-First Search explores one branch as deeply as possible before backtracking to another branch. For example, it opens the main folder, enters a subfolder, then a sub-subfolder, and continues until it finds report.pdf or reaches a dead end. A stack follows LIFO (Last In, First Out) and remembers which folders or branches remain to be explored. DFS may be implemented using an explicit stack or recursion.

(b) Why DFS uses less memory; space complexity

BFS stores many nodes in the frontier at the same time. DFS mainly stores the current path and the alternatives to revisit during backtracking. Its usual tree-search space complexity is O(bm), where b is the branching factor and m is the maximum depth.

(c) A long, roundabout search and optimality

If DFS explores a very deep branch first, it may examine many nested folders before finding report.pdf in a nearby branch. DFS does not guarantee the shortest path to a file; therefore, it is not optimal.

(d) Folder loops and the fix

If Folder A links to Folder B and Folder B links back to Folder A, plain DFS can revisit the same folders indefinitely. Use a visited set: mark each folder when first explored and skip it if it is encountered again. This prevents repeated exploration of cycles in a finite folder graph.

Q3. Uniform-Cost Search (UCS): Delivery Route Planning

Uniform-Cost Search expands the frontier node with the lowest cumulative path cost. It uses a priority queue. Road costs are S–A = 1, S–B = 4, A–B = 2, A–C = 5, B–C = 1, B–G = 6 and C–G = 3 km.

(a) UCS priority queue trace

Step	Expanded node (cost)	Priority queue after expansion (lowest cost first)
Initial	—	S(0)
1	S(0)	A(1), B(4)
2	A(1)	B(3), C(6)
3	B(3)	C(4), G(9)
4	C(4)	G(7)
5	G(7)	Empty

Explanation: Expanding A improves B from cost 4 to 3 and adds C at cost 6. Expanding B improves C to cost 4 and adds G at cost 9. Expanding C improves G to cost 7. When G is removed from the priority queue at cost 7, the cheapest route has been found.

(b) Cheapest route and total cost

Route: S → A → B → C → G

Total cost = 1 + 2 + 1 + 3 = 7 km.

(c) BFS route and its cost

With neighbors considered in the order given, BFS returns S → B → G because it reaches G through a two-edge path before a three-edge path. Its cost is 4 + 6 = 10 km. BFS minimizes the number of edges, not the total distance, so it does not return the cheapest route in this weighted graph.

(d) Optimality, completeness and zero/negative costs

UCS is optimal under standard conditions because it expands the lowest cumulative-cost path first; when the goal is removed from the priority queue, no cheaper path can be waiting under nonnegative edge costs and correct duplicate/update handling. UCS is complete when the branching factor is finite and every step cost is at least some positive ε. Zero-cost edges can be handled in a finite graph with suitable duplicate detection, but the usual positive-cost completeness guarantee no longer directly applies. Negative edges invalidate the standard optimality argument, and a negative-cost cycle can make a cheapest route undefined.

Q4. Depth-Limited Search (DLS): Customer-Support Chatbot Menu

(a) DLS and difference from DFS

Depth-Limited Search is DFS with a maximum depth limit l. It explores one branch at a time but does not expand nodes beyond the limit. Unlike plain DFS, it prevents unlimited descent beyond the chosen depth.

(b) Limits 3 and 4; cutoff and failure

The Refund Policy is at depth 4. With l = 3, DLS cannot reach it and returns a cutoff if the limit prevents further search. With l = 4, it can find the policy page if it is explored. A cutoff means unexplored nodes were stopped by the depth limit and a deeper goal may exist. Failure means the search exhausts all permitted paths without finding the goal and without being stopped by the limit.

(c) Time and space complexity

Time complexity: O(b^l). Space complexity: O(bl), where b is branching factor and l is the limit.

(d) Drawbacks, completeness and optimality

A limit that is too small can exclude the goal; a limit that is too large wastes time exploring unnecessary depth. DLS is complete for a finite-branching tree when the limit is at least the goal depth, but not if the goal lies beyond the limit. It is not generally optimal because it may find a deeper goal before a shallower one depending on branch order.

Q5. Iterative Deepening Search (IDS): Puzzle Solver on a Low-Memory Device

(a) Depth limits 0, 1, 2 and 3

Iteration	Nodes explored	Result
Limit 0	Root only	Goal not found
Limit 1	Root and children at depth 1	Goal not found
Limit 2	All nodes through depth 2	Goal not found
Limit 3	All nodes through depth 3	Goal found at depth 3

IDS runs depth-limited DFS repeatedly, increasing the limit by one each time. For b = 2 and a goal at depth 3, it searches limits 0, 1 and 2 without success, then finds the goal at limit 3. Assume the goal is in the explored part of the tree and children are considered in a fixed order.

(b) Why IDS combines BFS and DFS

IDS finds the shallowest goal like BFS when all actions have equal cost, while using memory closer to DFS because each iteration stores only a path and a limited number of alternatives. Re-exploring upper levels adds some work, but the memory savings are valuable when solution depth is unknown.

(c) Completeness, optimality and complexity

IDS is complete when the branching factor is finite and is optimal when all actions have equal cost. Time complexity is O(b^d), and space complexity is O(bd), where d is the shallowest goal depth.

Q6. Bidirectional Search: Connecting Two Cities

The question does not provide the actual city connections, so the following graph is an illustrative example only. Assume the undirected connections are A–B, A–C, B–D, C–E, D–G and E–G.

(a) Start searches from both ends

Run forward BFS from A and backward BFS from G. The forward frontier initially contains A, while the backward frontier initially contains G.

(b) Expand one level from each end

Search direction	Initial node	Frontier after one level
Forward BFS	A	B, C
Backward BFS	G	D, E

(c) Stop when the searches meet

Expanding the next levels, the forward search from B and C discovers D and E. The backward search from D and E discovers B and C. The searches have intersecting visited sets, so the route can be reconstructed through a meeting node.

(d) Reconstruct the shortest path

One shortest path in the illustrative graph is A → B → D → G, using 3 edges.

Another equally short route is A → C → E → G. Bidirectional BFS can reduce exploration by searching from both ends. Its typical time and space complexity are O(b^(d/2)) for balanced searches. It is complete and optimal for equal-cost edges under the usual BFS conditions.

Quick Revision Table

Algorithm	Data structure	Time complexity	Space complexity	Optimality
BFS	FIFO queue	O(b^(d+1))	O(b^(d+1))	Yes, equal costs
DFS	Stack / recursion	O(b^m)	O(bm)	Not generally
UCS	Priority queue	Depends on costs	Depends on costs	Yes, under standard assumptions
DLS	Depth-limited stack	O(b^l)	O(bl)	Not generally
IDS	Repeated depth-limited DFS	O(b^d)	O(bd)	Yes, equal costs
Bidirectional BFS	Two FIFO queues	O(b^(d/2)), typical	O(b^(d/2)), typical	Yes, equal costs

Notation: b = branching factor; d = shallowest goal depth; m = maximum search depth; l = depth limit. Complexities are standard search-tree estimates; graph-search implementations may differ due to duplicate detection.

Suggested Questions to Submit

The assessment asks for any TWO questions. Q1 (BFS) and Q3 (UCS) are recommended because they demonstrate search tracing, queue/priority-queue handling, path reconstruction, cost calculation, and theoretical explanation.